



JASHORE UNIVERSITY OF SCIENCE AND TECHNOLOGY

Department of Computer Science and Engineering

LAB REPORT

Course Code: CSE 3102

Course Title: Operating System and System Programming Lab

Lab No: 03

Lab Title: Implementation of Preemptive Shortest Job First (SJF) Scheduling Algorithm

Submitted To:

Monishanker Halder,
Assistant Professor
Dept. of Computer Science and Engineering, JUST

Submitted By:

Name: Adnan Zaman Niloy
Student ID: 210142
Session: 2021-2022
Year/Semester: 3rd Year, 1st Semester

Date of Submission: 2nd July 2025

Experiment Name : Implementation of Preemptive Shortest Job First (SJF) Scheduling Algorithm

1. Objective

To implement and analyze the Preemptive Shortest Job First (SJF), also known as Shortest Remaining Time First (SRTF), CPU scheduling algorithm and compute the waiting time, turnaround time, and average waiting time for a given set of processes.

2. Introduction and Theoretical Background

2.1 Problem Statement

Given the following set of processes with their arrival and burst times:

Process	Arrival Time	Burst Time
P1	0	12
P2	1	3
P3	2	5
P4	3	8
P5	4	2

Apply the **Shortest Remaining Time First (SRTF)** scheduling algorithm to:

- Generate the Gantt chart for execution order with preemption.
- Calculate each process's **completion**, **waiting**, and **turnaround** times.
- Determine the **average waiting time**.

2.2 Concept of Preemptive SJF (SRTF)

The Preemptive Shortest Job First (SJF), also known as Shortest Remaining Time First (SRTF), is a CPU scheduling algorithm that selects the process with the shortest remaining burst time among the available (arrived) processes. In the preemptive version:

- If a new process arrives with a shorter remaining time than the currently running process, the CPU preempts the current process and schedules the new one.
- It significantly reduces average waiting time and improves response time for shorter processes.
- However, it may cause starvation for longer processes if shorter jobs keep arriving.
- Frequent context switching may increase overhead in the system.

2.3 Algorithm Description

1. At each time unit, identify all **arrived and incomplete** processes.
2. Select the one with the **smallest remaining burst time**.
3. Execute the selected process for **1 unit of time**.
4. If the process finishes, calculate its **completion time, turnaround time, and waiting time**.
5. If no process is ready, increment the system clock.
6. Repeat until all processes are completed.

3. Methodology and Implementation

3.1 Data Structure

```
struct Process {  
    string id;  
    int arrival;  
    int burst;  
    int remaining;  
    int waiting = 0;  
    int turn = 0;  
    int completion = 0;  
    bool completed = false;  
};
```

3.2 Final C++ Code

```
#include <bits/stdc++.h>  
  
using namespace std;  
  
struct Process {  
    string id;  
    int arrival;  
    int burst;  
    int remaining;  
    int waiting = 0;  
    int turn = 0;
```

```

    int completion = 0;

    bool completed = false;
};

int main() {
    int n = 5;

    vector<Process> p(n);

    for (int i = 0; i < n; i++) {
        cin >> p[i].id >> p[i].arrival >> p[i].burst;

        p[i].remaining = p[i].burst;
    }

    int complete = 0, current_time = 0;

    float total_wait = 0, total_turn = 0;

    while (complete < n) {
        int shortest = -1;

        int min_remain = INT_MAX;

        for (int i = 0; i < n; i++) {
            if (!p[i].completed && p[i].arrival <= current_time && p[i].remaining < min_remain) {
                min_remain = p[i].remaining;

                shortest = i;
            }
        }

        if (shortest == -1) {
            current_time++;

            continue;
        }

        p[shortest].remaining--;

        current_time++;

        if (p[shortest].remaining == 0) {
            p[shortest].completion = current_time;

            p[shortest].turn = p[shortest].completion - p[shortest].arrival;

```

```

        p[shortest].waiting = p[shortest].turn - p[shortest].burst;
        p[shortest].completed = true;

        total_wait += p[shortest].waiting;
        total_turn += p[shortest].turn;
        complete++;
    }
}

cout << "Process\tArrival\tBurst\tCompletion\tWaiting\tTurn" << endl;
for (int i = 0; i < n; i++) {
    cout << p[i].id << "\t"
        << p[i].arrival << "\t"
        << p[i].burst << "\t"
        << p[i].completion << "\t\t"
        << p[i].waiting << "\t"
        << p[i].turn << endl;
}

cout << "\nAverage Waiting Time: " << total_wait / n << endl;
return 0;
}

```

4. Test Case Analysis

4.1 Input

p1	0	12
p2	1	3
p3	2	5
p4	3	8
p5	4	2

4.2 Gantt Chart Execution Timeline

Execution Order Explanation:

- Time 0–1: P1 starts execution (only process available at time 0).
- Time 1–4: P2 arrives and preempts P1 (shorter remaining time).

- Time 4–6: P5 arrives and preempts P2 (even shorter time).
- Time 6–11: P3 becomes the shortest and executes fully.
- Time 11–19: P4 executes (shorter than remaining P1).
- Time 19–30: P1 resumes and finishes its remaining time.

Grant Chart:

0	1	4	6	11	19	30
p1	p2	P5	P3	P4	P1	

4.3 Output Table

Process	Arrival	Burst	Completion	Waiting	Turnaround
P1	0	12	30	18	30
P2	1	3	4	0	3
P3	2	5	11	4	9
P4	3	8	19	8	16
P5	4	2	6	0	2

4.4 Average Waiting Time

Avg. Waiting Time = $18+0+4+8+0=30/5=6.00$

5. Results and Discussion

- P2 and P5 complete quickly after arriving due to their short burst times.
- P1, although arriving first, is preempted several times, causing a waiting time of 18.
- The average waiting time of 6.00 demonstrates the effectiveness of SRTF in reducing wait time compared to non-preemptive scheduling.
- Preemption provides better response for short jobs but may delay longer ones.

6. Conclusion

The Preemptive Shortest Job First (SJF) algorithm improves CPU scheduling performance by minimizing average waiting time through preemption. In this simulation, it achieved a lower average wait time of 6.00 units. While this favors short processes, mechanisms like aging may be needed in real systems to prevent starvation of longer tasks.